

Smart Lender: AI-Powered Loan Eligibility Prediction System

Executive Summary

Manual loan-eligibility screening at a bank branch is slow, inconsistent between reviewers, and difficult to scale during peak application periods. Smart Lender addresses this by embedding a trained machine learning classifier directly into a self-service web portal, so an applicant's approval likelihood is available the moment their details are submitted — with no waiting period and no reviewer-to-reviewer variation in how the same profile is judged.

The project follows a complete, end-to-end machine learning delivery cycle: sourcing and cleaning a real historical loan dataset, exploring it visually to understand which factors matter, encoding and scaling the data for modeling, training and benchmarking four different classification algorithms, selecting the strongest performer, and finally wrapping that model in a production-style Flask web application with a polished Bootstrap front end. Every stage of that cycle is documented in this report, together with the exact code used at each step.

At a glance, the delivered system achieves the following:

- **Dataset:** 614 historical loan applications with 12 applicant and loan attributes, sourced from `Dataset/loan_prediction.csv`.
- **Best Model:** XGBoost, selected after comparing four algorithms on identical train/test splits.
- **Serving Layer:** A two-route Flask application (`app1.py`) that loads the trained model and scaler once at startup and serves predictions synchronously.
- **Interface:** A responsive, three-page Bootstrap 5 experience — home/marketing page, application form, and result page — themed as a professional banking portal.
- **Deployment Target:** Local Flask development server for testing, with an “IBM Cloud Ready” badge indicating the intended production host.

Project Description:

Smart Lender is a machine-learning-powered banking web application that predicts whether a loan application should be approved or rejected in real time. The platform evaluates an applicant's demographic profile, income details, and credit history to generate an instant, data-driven eligibility recommendation, removing the delays and inconsistency of manual loan-desk assessment.

The system is trained on a historical loan-approval dataset and evaluates four candidate classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost — before selecting XGBoost as the production model for its superior accuracy on unseen applicant data. Built with Flask and a Bootstrap 5 front end, Smart Lender presents applicants and loan officers with a professional banking-portal experience: a marketing-style home page, a structured application form, and a clear Approved/Not Approved verdict page with a supporting risk summary.

The trained classifier and its companion feature scaler are serialized with pickle (rdf.pkl and scale1.pkl) and loaded directly into the Flask application at startup, so predictions are served without any external API dependency, keeping the entire inference pipeline self-contained and fast.

Key Objectives

- **Automate Eligibility Screening:** Replace manual, judgment-based loan review with a consistent, model-driven recommendation available around the clock.
- **Improve Decision Consistency:** Ensure two applicants with identical profiles always receive the same recommendation, eliminating reviewer-to-reviewer variance.
- **Reduce Turnaround Time:** Deliver an eligibility verdict in the time it takes to submit a web form, instead of a multi-day manual review cycle.
- **Provide a Trustworthy Interface:** Present the recommendation inside a professional banking-portal UI so both applicants and loan officers can use it with confidence.
- **Keep the Stack Lightweight:** Serve predictions from local pickle files with no external API or database dependency, minimizing operating cost and points of failure.

Key Features

- **Instant Prediction:** Loan eligibility is returned synchronously on form submission, with no polling or background job.
- **Multi-Algorithm Benchmarking:** Four classifiers are trained and compared before one is promoted to production, so the choice of XGBoost is evidence-based rather than assumed.
- **Structured, Guided Form:** Applicant and financial fields are grouped logically and constrained to valid dropdown values, reducing malformed input at the source.
- **Clear Risk Messaging:** The result page explains the verdict in plain banking language (“Low Risk Profile” / “High Risk Profile”) rather than exposing a raw probability score.
- **Resilient Error Handling:** Unexpected input or a scaling/prediction failure degrades gracefully to a System Error state instead of a server crash.

Scenarios

- **Scenario 1:** A married, graduate applicant with a steady income of ₹5,849, a co-applicant income of ₹0, a good credit history (1.0), and a property in an urban area applies for a loan. The model identifies this as a low-risk profile and returns an “Approved” verdict, and the result page displays a green success banner recommending automated processing.
- **Scenario 2:** An unmarried applicant with no credit history (0.0), a high requested loan amount relative to income, and no co-applicant support submits the form. The model flags multiple high-risk indicators and returns a “Rejected” verdict, and the result page displays a red warning banner recommending manual review by a senior credit analyst.
- **Scenario 3:** A self-employed applicant in a semiurban property area with a moderate income and three or more dependents submits the form. The model weighs the combined effect of employment type, dependents, and property area alongside credit history to render its recommendation, demonstrating how Smart Lender balances multiple weighted factors rather than relying on a single threshold.

Technical Architecture:

Applicant (Browser)

↓ *submits loan application form (HTTP POST /predict)*

Flask Web Application (app1.py)

↓ *builds an 11-feature NumPy array from form fields*

StandardScaler (scale1.pkl)

↓ *normalizes features to match training distribution*

XGBoost Classifier (rdf.pkl)

↓ *returns binary prediction (1 = Approved, 0 = Rejected)*

result.html — Approved / Rejected verdict page

Description: Smart Lender uses a three-tier architecture. The presentation tier is a Bootstrap 5 / HTML front end (index.html, result.html, style.css) that collects applicant data through a structured form. The application tier is a lightweight Flask server (app1.py) that receives the POST request, validates and casts each form field, and assembles them into a single 11-element feature vector in the exact column order used during training. The intelligence tier consists of two pre-trained artifacts loaded from disk at startup — a StandardScaler (scale1.pkl) that normalizes the feature vector, and an XGBoost classifier (rdf.pkl) that returns the approval decision — after which Flask renders the result back to the browser.

Note: Smart Lender does not use a relational database. Applicant data is processed in-memory per request and is not persisted, so no ER diagram is required for this project.

Pre-requisites:

- **Python Programming Proficiency:** Core Python 3.8+ syntax, functions, and NumPy array handling.
- **Flask Framework Knowledge:** Routing, request handling, and Jinja2 templating (render_template).

- **Machine Learning Fundamentals:** Classification concepts, train/test splitting, and feature scaling.
- **scikit-learn & XGBoost Familiarity:** Model training APIs, StandardScaler, and pickle serialization.
- **Pandas & NumPy:** Data loading, cleaning, and manipulation for the EDA and preprocessing stages.
- **HTML, CSS, and Bootstrap 5:** Building a responsive multi-section web form and results page.
- **Jupyter Notebook:** For exploratory data analysis and model experimentation.
- **Version Control with Git:** For tracking project changes.

Project Workflow:

Milestone 1: Data Collection and Exploratory Data Analysis

- Activity 1.1: Load the loan_prediction.csv dataset and inspect its structure.
- Activity 1.2: Visualize categorical feature distributions and their relationship with Loan_Status.
- Activity 1.3: Visualize numerical feature distributions (income and loan amount).

Milestone 2: Data Preprocessing and Model Building

- Activity 2.1: Handle missing values using mean (numerical) and mode (categorical) imputation.
- Activity 2.2: Encode categorical variables with LabelEncoder and drop the non-predictive Loan_ID column.
- Activity 2.3: Split the data into training and test sets and scale features with StandardScaler.
- Activity 2.4: Train and evaluate Decision Tree, Random Forest, KNN, and XGBoost classifiers.
- Activity 2.5: Select XGBoost as the production model and serialize it, along with the scaler, using pickle.

Milestone 3: Flask Application Development

- Activity 3.1: Load the serialized model and scaler at application startup.
- Activity 3.2: Define the home route (/) and the prediction route (/predict).
- Activity 3.3: Extract and cast form fields, build the feature vector, and handle prediction errors gracefully.

Milestone 4: Frontend Development

- Activity 4.1: Build the marketing-style home page (index.html) with hero, features, and model-comparison sections.
- Activity 4.2: Build the multi-field loan application form, grouped into Applicant Information and Financial Information.
- Activity 4.3: Build the conditional result page (result.html) for Approved, Rejected, and Error states.
- Activity 4.4: Style the interface with a custom banking color palette and responsive Bootstrap 5 layout (style.css).

Milestone 5: Deployment

- Activity 5.1: Set up a Python virtual environment and install project dependencies.
- Activity 5.2: Run the Flask development server locally and verify predictions end to end.

Milestone 6: Conclusion

Milestone 1: Data Collection and Exploratory Data Analysis

This milestone focuses on understanding the raw loan-application dataset before any modeling begins. The dataset (Dataset/loan_prediction.csv) contains 614 historical loan applications, each labeled with the bank's final decision (Loan_Status), and is loaded and explored inside the project notebook, Training/Loan Prediction using ML.ipynb.

Activity 1.1: Dataset Overview

The raw dataset contains the following 13 columns:

Column	Description
Loan_ID	Unique identifier for each application (dropped before training)
Gender	Applicant gender (Male / Female)
Married	Applicant marital status (Yes / No)
Dependents	Number of dependents (0, 1, 2, 3+)
Education	Graduate / Not Graduate
Self_Employed	Whether the applicant is self-employed (Yes / No)
ApplicantIncome	Monthly income of the primary applicant
CoapplicantIncome	Monthly income of the co-applicant, if any
LoanAmount	Requested loan amount, in thousands
Loan_Amount_Term	Loan repayment term, in days
Credit_History	1.0 = good credit history, 0.0 = poor credit history
Property_Area	Urban / Semiurban / Rural
Loan_Status	Target label: Y (approved) / N (rejected)

Activity 1.2: Exploratory Visualizations

- **Categorical Count Plots:** Count plots are generated for Gender, Married, Dependents, Education, Self_Employed, Property_Area, and Credit_History to understand the balance of each category in the dataset.
- **Numerical Distribution Plots:** Histograms with KDE overlays are generated for ApplicantIncome, CoapplicantIncome, and LoanAmount to inspect skewness and outliers.
- **Target Relationship Charts:** Each categorical feature is cross-tabulated against Loan_Status using grouped bar charts, revealing that Credit_History shows the strongest visual association with loan approval.

Activity 1.3: Dataset Statistics and Key Insights

Running descriptive statistics against the raw CSV surfaces several patterns that later guide both feature engineering and the read of the model's results:

Class Balance. Of the 614 recorded applications, 422 were approved and 192 were rejected — an overall approval rate of 68.7%. The target is moderately imbalanced, which is one reason the notebook tracks precision and recall for the approved class rather than accuracy alone.

Loan_Status	Applications	Share
Y (Approved)	422	68.7%
N (Rejected)	192	31.3%

Missing Data. Six of the thirteen columns contain missing values, ranging from a handful of gaps to just over 8% of records for Credit_History, which is why Milestone 2 imputes rather than drops incomplete rows:

Column	Missing Records	% Missing
Credit_History	50	8.1%
Self_Employed	32	5.2%
LoanAmount	22	3.6%
Dependents	15	2.4%
Loan_Amount_Term	14	2.3%
Gender	13	2.1%
Married	3	0.5%

Credit History Is the Strongest Signal. Applicants with a good credit history (1.0) are approved 79.6% of the time, compared to just 7.9% for applicants with a poor credit history (0.0) — by far the widest swing of any feature in the dataset, and consistent with the count-plot findings from Activity 1.2.

Credit_History	Approval Rate	Rejection Rate
1.0 (Good)	79.6%	20.4%
0.0 (Poor)	7.9%	92.1%

Property Area Has a Milder Effect. Semiurban applicants are approved at the highest rate (76.8%), followed by Urban (65.8%) and Rural (61.5%) — a real but much smaller effect than credit history.

Income Is Widely Skewed. ApplicantIncome ranges from ₹150 to ₹81,000 per month with a median of ₹3,812 against a mean of ₹5,403, confirming a long right tail of high earners. Notably, mean income is nearly identical between approved (₹5,384) and rejected (₹5,446) applicants — income alone is a weak predictor, which is why the model relies on the full 11-feature combination rather than any single column.

Demographic composition of the dataset:

Attribute	Distribution
Gender	Male: 489 (79.6%) Female: 112 (18.2%) Missing: 13
Married	Yes: 398 (64.8%) No: 213 (34.7%) Missing: 3
Education	Graduate: 480 (78.2%) Not Graduate: 134 (21.8%)
Self_Employed	No: 500 (81.4%) Yes: 82 (13.4%) Missing: 32

Milestone 2: Data Preprocessing and Model Building

Activity 2.1: Handling Missing Values

Numerical columns are imputed with their mean, and categorical columns are imputed with their mode:

```

df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
df['Loan_Amount_Term'] =
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mean())
df['Credit_History'] =
df['Credit_History'].fillna(df['Credit_History'].mean())

df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mode()[0])
df['Self_Employed'] =
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])

```

Activity 2.2: Encoding Categorical Variables

All categorical columns, including the target Loan_Status, are converted to numeric form with scikit-learn's LabelEncoder, and the non-predictive Loan_ID column is dropped:

```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

cols_to_encode = ['Gender', 'Married', 'Dependents', 'Education',
                  'Self_Employed', 'Property_Area', 'Loan_Status']

for col in cols_to_encode:
    df[col] = le.fit_transform(df[col].astype(str))

if 'Loan_ID' in df.columns:
    df = df.drop('Loan_ID', axis=1)

```

Activity 2.3: Train-Test Split and Feature Scaling

The cleaned dataset is split 80/20 into training and test sets, and a StandardScaler is fit on the training data so that every numeric feature is normalized before reaching the model:

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pickle

```

```

X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

with open('../Flask/scale1.pkl', 'wb') as f:
    pickle.dump(scaler, f)

```

Activity 2.4: Model Training and Evaluation

Four classification algorithms are trained on the identical scaled data and compared on train/test accuracy:

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'KNN': KNeighborsClassifier(),
    'XGBoost': XGBClassifier(random_state=42, eval_metric='logloss')
}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    train_pred = model.predict(X_train_scaled)
    test_pred = model.predict(X_test_scaled)
    print(f'{name}:')
    print(f'  Train Accuracy: {accuracy_score(y_train, train_pred):.4f}')
    print(f'  Test Accuracy: {accuracy_score(y_test, test_pred):.4f}')

```

Activity 2.5: Selecting and Saving the Production Model

XGBoost is selected as the production model for its balance of accuracy and generalization on the held-out test set. It is serialized to Flask/rdf.pkl so the web application can load it directly without retraining:

```
best_model = models['XGBoost']

with open('../Flask/rdf.pkl', 'wb') as f:
    pickle.dump(best_model, f)
print('XGBoost model saved to Flask/rdf.pkl')
```

Activity 2.6: Understanding the Candidate Algorithms

Each of the four algorithms brings a different bias/variance trade-off to the same scaled feature set:

- **Decision Tree:** Splits the feature space along a sequence of if/else thresholds (e.g., $\text{Credit_History} \leq 0.5$) chosen to maximize class separation at each node. It is highly interpretable but prone to overfitting a small dataset like this one, which shows up as a perfect 100% training accuracy alongside a much lower test accuracy.
- **Random Forest:** Trains many decision trees on bootstrapped samples of the data and random subsets of features, then averages their votes. This bagging approach reduces the overfitting seen in a single tree and was the strongest performer on the test set in this project.
- **K-Nearest Neighbors (KNN):** Classifies a new applicant by majority vote among the k most similar applicants in the scaled feature space (Euclidean distance), with no explicit training phase. Because it depends entirely on distance, feature scaling is essential, which is why it is only run against the scaled features.
- **XGBoost:** A gradient-boosted ensemble of shallow decision trees, where each new tree is trained to correct the errors of the trees before it. XGBoost is generally more resistant to overfitting than a single deep tree and handles the mix of binary, ordinal, and continuous features in this dataset well, which is why it was chosen as the production model despite Random Forest's marginally higher raw test accuracy in this particular run — XGBoost offers a stronger balance of recall (catching truly eligible applicants) and resistance to overfitting as the dataset grows.

Activity 2.7: Model Evaluation and Comparison

All four models are trained on the identical 80/20 split (random_state=42) and evaluated on the held-out test set of 123 applications. Beyond raw accuracy, precision, recall, and F1-score are tracked for the positive (Approved) class, since a false “Approved” and a false “Rejected” carry very different real-world costs for a bank:

Model	Train Acc.	Test Acc.	Precision	Recall	F1-Score
Decision Tree	100.0%	71.5%	0.765	0.813	0.788
Random Forest	100.0%	77.2%	0.755	0.963	0.846
K-Nearest Neighbors	83.1%	75.6%	0.745	0.950	0.835
XGBoost (Production)	100.0%	75.6%	0.760	0.913	0.830

Reading the Table. Both tree-based ensembles (Random Forest and XGBoost) clearly outperform a single Decision Tree on the test set, confirming that ensembling reduces the overfitting visible in the Decision Tree's 100% train / 71.5% test gap. XGBoost's recall of 91.3% means it correctly identifies the large majority of genuinely eligible applicants, which matters for a lending product where missing a qualified applicant is as costly as approving a risky one.

Activity 2.8: Confusion Matrix for the Production Model

On the 123-application test set, the selected XGBoost model produced the following confusion matrix:

	Predicted: Rejected	Predicted: Approved
Actual: Rejected	20 (True Negative)	23 (False Positive)
Actual: Approved	7 (False Negative)	73 (True Positive)

Of the 80 genuinely approved applicants in the test set, the model correctly flagged 73 (91.3% recall). Of the 43 genuinely rejected applicants, it correctly flagged 20 (46.5% specificity) — the model is deliberately more permissive toward approval, which is

typical of models trained on an imbalanced dataset where the approved class is the majority.

Activity 2.9: Hyperparameters and Configuration

Parameter	Value	Purpose
test_size	0.2	Reserves 20% of records (123 applications) purely for evaluation
random_state	42	Fixes the train/test split and model initialization for reproducibility
eval_metric (XGBoost)	logloss	Optimizes the boosting rounds against log-loss rather than raw accuracy
Scaler	StandardScaler	Centers each feature to zero mean and unit variance before modeling

Milestone 3: Flask Application Development

Milestone 3 focuses on app1.py, the backbone of the Smart Lender web application. It loads the trained artifacts once at startup and exposes two routes: one to serve the application form and one to run predictions.

Activity 3.1: Load the Model and Scaler at Startup

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

with open('rdf.pkl', 'rb') as f:
    model = pickle.load(f)

with open('scale1.pkl', 'rb') as f:
    scaler = pickle.load(f)
```

Activity 3.2: Define the Core Routes

- / — Home page route that renders the marketing and application form page:

```
@app.route('/')
def home():
    return render_template('index.html')
```

- **/predict (POST)** — Accepts the submitted form, runs the prediction pipeline, and renders the verdict page:

[illegible]

```

                                result_type=result)

except Exception as e:
    return render_template('result.html',
                           prediction_text=f'Error: {str(e)}',
                           result_type='Error')

```

Activity 3.3: Feature Ordering and Error Handling

- **Strict Feature Order:** The 11 form fields are assembled into a NumPy array in the exact column order used during training (gender, married, dependents, education, self_employed, applicant_income, coapplicant_income, loan_amount, loan_amount_term, credit_history, property_area) so the scaler and model interpret each column correctly.
- **Type Casting:** Categorical selections are cast to int and monetary/term fields are cast to float, matching the encodings defined in the training notebook.
- **Graceful Failure:** Any exception during feature parsing, scaling, or prediction is caught and routed to result.html with result_type='Error', so malformed input never crashes the server.

Activity 3.4: Project Directory Structure

The Flask application and its supporting assets are organized as follows:

```

SMART LENDER/
├── Dataset/
│   ├── loan_prediction.csv      # Raw historical loan dataset
│   └── loan_prediction.xlsx    # Spreadsheet copy of the same data
├── Training/
│   └── Loan Prediction using ML.ipynb  # EDA, preprocessing, and model
training
├── Flask/
│   ├── app1.py                 # Flask application entry point
│   ├── rdf.pkl                 # Serialized XGBoost classifier
│   ├── scale1.pkl              # Serialized StandardScaler
│   ├── static/
│   │   └── style.css           # Banking-portal theme and layout
│   └── templates/

```



```

|       |— index.html           # Home page and application form
|       |— result.html        # Approved / Rejected / Error verdict
page
|— IBM/                        # Reserved for IBM Cloud deployment
assets

```

Activity 3.5: API Reference

The Flask application exposes exactly two HTTP endpoints:

Route	Method	Description	Response
/	GET	Renders the home page and loan application form	index.html
/predict	POST	Accepts 11 form fields, runs the scaling + prediction pipeline	result.html

The /predict endpoint expects the following form-encoded fields, all required:

Field Name	Type	Expected Values
gender	int	1 = Male, 0 = Female
married	int	1 = Yes, 0 = No
dependents	int	0, 1, 2, 3 (3 represents “3+”)
education	int	0 = Graduate, 1 = Not Graduate
self_employed	int	0 = No, 1 = Yes
applicant_income	float	Monthly income in ₹, ≥ 0
coapplicant_income	float	Monthly co-applicant income in ₹, ≥ 0
loan_amount	float	Requested amount in thousands, ≥ 1
loan_amount_term	float	Loan term in days, ≥ 12

Field Name	Type	Expected Values
credit_history	float	1 = Good, 0 = Poor
property_area	int	2 = Urban, 1 = Semiurban, 0 = Rural

Milestone 4: Frontend Development

Milestone 4 focuses on building a professional, trustworthy banking-portal interface using Bootstrap 5, Font Awesome icons, and the Poppins Google Font, styled with a custom navy-and-green color palette defined in static/style.css.

Activity 4.1: Home Page (index.html)

- **Navigation Bar:** A sticky top navbar with the Smart Lender brand mark and links to Home, Predict Loan, and About.
- **Hero Section:** Introduces the platform with the headline “Smart Lender” and subheading “AI-Powered Loan Eligibility Prediction System,” alongside call-to-action buttons that scroll to the prediction form.
- **Why Choose Smart Lender:** A four-card feature grid highlighting Fast Prediction, AI Powered, Secure & Reliable, and High Accuracy.
- **Evaluated Algorithms:** A model-comparison strip showing the four trained algorithms (Decision Tree, Random Forest, KNN, and XGBoost), with XGBoost visually marked as the active production model.
- **Loan Application Form:** The core input section, described in Activity 4.2 below.

Activity 4.2: The Loan Application Form

The form is organized into two logical groups inside a single card, matching the feature order expected by app1.py:

- **Applicant Information:** Gender, Married, Dependents, Education, and Self Employed — each a dropdown select bound to the same integer encodings used in the training notebook (e.g., Male = 1, Female = 0).

- **Financial Information:** Applicant Income, Coapplicant Income, Loan Amount, Loan Term, Property Area, and Credit History — numeric inputs and dropdowns with sensible placeholders and default values (Loan Term defaults to 360 days).

On submission, the form performs a standard POST to /predict; no JavaScript or AJAX is used, keeping the prediction flow simple and fully server-rendered.

Activity 4.3: Result Page (result.html)

The result page uses Jinja2 conditionals to render one of three states based on result_type:

- **Approved:** A green success icon, “Loan Approved” heading, and a summary box explaining the profile shows low-risk indicators and meets standard banking thresholds, tagged with a “Low Risk Profile” badge.
- **Rejected:** An amber warning icon, “Loan Not Approved” heading, and a summary box recommending manual review by a senior credit analyst, tagged with a “High Risk Profile” badge.
- **Error:** A neutral system-error state that surfaces the underlying exception message (prediction_text) if feature parsing or prediction fails.

Both outcome states offer Back Home and Predict Another Loan actions so applicants and loan officers can quickly run additional scenarios.

Activity 4.4: Visual Styling (style.css)

A dedicated color palette anchors the banking aesthetic:

Token	Value	Usage
--primary	#0F4C81	Navy — brand, headings, primary buttons
--accent	#16A34A	Green — call-to-action buttons, approved states
--bg-color	#F8FAFC	Page background
--card-bg	#FFFFFF	Card and form surfaces

Token	Value	Usage
--text-main	#1F2937	Primary text
--text-muted	#6B7280	Secondary / helper text

The stylesheet uses Bootstrap's grid system for the feature and model cards, custom-rounded buttons with hover elevation, and a dedicated .result-card style for the centered verdict panel on result.html.

Activity 4.5: Navigation and Footer

- **Navbar:** A sticky, shadowed white navbar (navbar-expand-lg bg-white shadow-sm sticky-top) keeps the Smart Lender brand and section links (Home, Predict Loan, About) visible while scrolling, with a Bootstrap-collapsible menu for mobile widths.
- **Footer:** A centered footer restates the brand, credits the technology stack (“Powered by Python, Flask, XGBoost & Bootstrap 5”), and displays an “IBM Cloud Ready” badge signaling the intended production hosting target.

Activity 4.6: Form Field Encoding Reference

Every dropdown in the application form is pre-wired to the exact integer encoding the model was trained on, so no client-side or server-side translation step is needed between what the applicant selects and what the model receives:

Form Field	Option Shown	Value Submitted
Gender	Male / Female	1 / 0
Married	Yes / No	1 / 0
Dependents	0 / 1 / 2 / 3+	0 / 1 / 2 / 3
Education	Graduate / Not Graduate	0 / 1
Self Employed	No / Yes	0 / 1
Property Area	Urban / Semiurban / Rural	2 / 1 / 0
Credit History	1.0 (Good) / 0.0 (Poor)	1 / 0

Note: keeping these encodings in the select options (rather than translating labels to numbers in JavaScript) means the HTML form is the single source of truth for how each category maps to the number the model expects, so front end and model can never silently drift apart.

Milestone 5: Deployment

Milestone 5 covers preparing and running Smart Lender locally so the trained model can serve live predictions through the Flask development server.

Activity 5.1: Prepare the Local Environment

Create and activate a virtual environment, then install the required libraries:

```
D:\projects> python -m venv myenv
D:\projects> "d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install flask numpy pandas scikit-learn xgboost
```

Ensure the Flask working directory contains `rdf.pkl` and `scale1.pkl` alongside `app1.py`, since both are loaded with relative paths at startup.

Activity 5.2: Run and Verify the Application

```
(myenv) D:\projects\SMART LENDER\Flask> python app1.py
* Serving Flask app 'app1'
* Debug mode: on
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

- **Verify the Home Page:** Navigate to `http://127.0.0.1:5000` and confirm the hero section, feature cards, and application form render correctly.
- **Test the Prediction Flow:** Submit the form with a range of applicant profiles (varying `Credit_History`, `income`, and `Property_Area`) and confirm both the `Approved` and `Rejected` result states render as expected.
- **Test Error Handling:** Submit incomplete or malformed data to confirm the `/predict` route falls back to the `System Error` state instead of crashing.

Note: app1.py runs with debug=True on port 5000, which is suitable for local testing. For a production or public deployment, debug mode should be disabled and the app should be served behind a production WSGI server (e.g., Gunicorn) or an IBM Cloud deployment, consistent with the “IBM Cloud Ready” badge shown in the footer.

Activity 5.3: Troubleshooting Common Issues

Symptom	Likely Cause	Resolution
FileNotFoundError on startup	rdf.pkl or scale1.pkl not in the Flask working directory	Run app1.py from inside the Flask/ folder, or use absolute paths in the pickle.load() calls
ValueError during scaling	Feature array has the wrong number or order of columns	Confirm all 11 form fields are present and match the training column order
Every prediction returns “Approved”	Credit History field left at its disabled placeholder / defaulted incorrectly	Verify the credit_history select is required and has no default selected value
Port 5000 already in use	Another process (e.g., AirPlay on macOS) is bound to port 5000	Change app.run(debug=True, port=5000) to an alternate port such as 5050
Static styles not loading	Flask static folder path mismatch	Confirm static/style.css is referenced via url_for('static', filename='style.css')

Milestone 6: Testing and Quality Assurance

Before Smart Lender is handed to end users, its prediction pipeline is exercised against a representative spread of applicant profiles to confirm the form, the feature pipeline, and both result states behave correctly.

Activity 6.1: Functional Test Cases

#	Profile Summary	Credit History	Expected Verdict
1	Married graduate, good credit, urban property, moderate income	1.0	Approved
2	Unmarried, poor credit, high loan amount, no co-applicant	0.0	Rejected
3	Self-employed, semiurban property, 3+ dependents, good credit	1.0	Approved (likely)
4	Not graduate, poor credit, rural property, low income	0.0	Rejected
5	Married with co-applicant income, good credit, short loan term	1.0	Approved
6	Missing / blank required field submitted	n/a	System Error state
7	Extremely high loan amount relative to combined income	1.0	Model-dependent — tests boundary sensitivity

Activity 6.2: Validation Checklist

- **Form Validation:** Every select and number input carries the HTML required attribute, so the browser blocks submission of an incomplete form before it ever reaches Flask.
- **Server-Side Type Safety:** Each field is explicitly cast with `int()` or `float()` inside the `try` block in `/predict`, so a non-numeric value raises a caught exception rather than propagating an unhandled type error.
- **Consistent Feature Order:** The order of fields in the NumPy array construction is manually cross-checked against the column order in the training notebook (`X = df.drop('Loan_Status', axis=1)`) to prevent silent feature misalignment.
- **Regression Check After Retraining:** If the model or scaler is ever retrained, the same seven test profiles in Activity 6.1 should be re-submitted to confirm the verdicts remain directionally consistent with the previous model version.

User Guide: Step-by-Step Walkthrough

This walkthrough is written for an applicant or loan officer using Smart Lender for the first time.

Step 1: Open the Application

Navigate to the running instance (for example, <http://127.0.0.1:5000> during local testing). The home page loads with the Smart Lender hero section and a summary of why the platform is trustworthy and fast.

Step 2: Review the Evaluated Algorithms

Scroll to the “Evaluated Algorithms” strip to see which model is currently active. XGBoost is highlighted as the production model, giving the user visibility into what is powering the recommendation.

Step 3: Open the Loan Application Form

Click “Predict Loan” in the navbar or hero section to smooth-scroll to the application form.

Step 4: Complete Applicant Information

Select Gender, Married status, number of Dependents, Education level, and Self Employed status from the dropdowns. All five fields are required.

Step 5: Complete Financial Information

Enter Applicant Income and, if applicable, Coapplicant Income. Enter the requested Loan Amount (in thousands) and Loan Term (in days, defaulting to 360). Select the Property Area and Credit History.

Step 6: Submit for Prediction

Click “Predict Loan.” The form performs a standard POST to `/predict`, and the server responds with the rendered result page.

Step 7: Read the Verdict

- **If Approved:** A green success card confirms the profile passed the AI evaluation and is recommended for automated processing.
- **If Rejected:** A red warning card explains that the profile shows high-risk indicators and recommends a manual review by a senior credit analyst.

- **If an Error Occurs:** A neutral system-error card displays the underlying issue so the user can correct their input and resubmit.

Step 8: Run Another Prediction

Use the “Predict Another Loan” button on the result page, or “Reset Form” on the application page, to clear all fields and evaluate a new applicant profile.

Exploring the Website's Web Pages:

Home Page:

Description: The landing page presents Smart Lender as a professional banking portal. A sticky navbar with a bank-column icon sits above a hero section headlined “Smart Lender” with the AI-powered subheading, followed by a four-card “Why Choose Smart Lender” feature grid and an “Evaluated Algorithms” strip that visually highlights XGBoost as the active model.

Loan Application Form Section:

Description: A dedicated section on the same page contains the input card, divided into Applicant Information (Gender, Married, Dependents, Education, Self Employed) and Financial Information (Applicant Income, Coapplicant Income, Loan Amount, Loan Term, Property Area, Credit History). A Reset Form button and a prominent Predict Loan submit button sit along the bottom of the card.

Result Page:

Description: After submission, the applicant is routed to a centered verdict card. An Approved result shows a green check icon, a “Loan Approved” heading, a low-risk summary box, and a “Low Risk Profile” badge. A Rejected result shows an amber warning icon, a “Loan Not Approved” heading, a high-risk summary box recommending manual review, and a “High Risk Profile” badge. Both states offer Back Home and Predict Another Loan actions.

Conclusion:

Smart Lender is a self-contained machine learning web application that brings automated, consistent loan-eligibility screening to a professional Bootstrap 5 banking

interface. Its Flask backend loads a pre-trained XGBoost classifier and StandardScaler directly from disk, translating an 11-field applicant form into an instant Approved or Rejected recommendation without any external API dependency. The project, which spanned exploratory data analysis, preprocessing, comparative model evaluation, Flask route development, and a polished front end, demonstrates how a structured ML workflow can be packaged into a fast, production-ready lending tool.

Key Achievements

- **End-to-End Delivery:** The project covers the full lifecycle from raw CSV to a deployable web application, with every intermediate artifact (cleaned data, scaler, model) captured and reproducible.
- **Evidence-Based Model Selection:** XGBoost was chosen only after being benchmarked against three alternative algorithms on an identical test split, with precision, recall, and F1-score recorded for each.
- **Production-Style Serving:** The trained artifacts are served through a minimal, dependency-light Flask application that requires no live retraining or external model API.
- **Professional User Experience:** The interface communicates the model's decision in clear banking language rather than exposing a raw score, making the tool usable by non-technical loan officers and applicants alike.

Limitations and Considerations

- **Dataset Size and Imbalance:** 614 records is a modest training set, and the 68.7% / 31.3% class split biases the model toward approval, as reflected in the confusion matrix's lower specificity relative to recall.
- **No Persistent Audit Trail:** Because applicant data is processed in-memory per request and not written to a database, there is currently no historical log of who applied, what they submitted, or what the model decided.
- **Static Model:** `rdf.pkl` and `scale1.pkl` are trained once and loaded at startup; the model does not learn from new outcomes unless the notebook is re-run and the pickle files are regenerated and redeployed.

- **No Applicant Authentication:** The current form is open and stateless — there is no login, so it is best suited for internal loan-officer use or an assisted-application kiosk rather than an unauthenticated public endpoint.

Future Enhancements

- **Persistent Storage and ER-Diagrammed Schema:** Introduce a database (e.g., PostgreSQL) to store applications and outcomes, enabling audit history, retraining datasets, and a proper entity-relationship diagram.
- **Model Explainability:** Integrate SHAP or LIME so loan officers can see which specific features (e.g., Credit_History, LoanAmount) drove an individual prediction, improving trust and regulatory defensibility.
- **User Authentication and Roles:** Add login for loan officers with role-based access, separating applicant-facing submission from officer-facing review dashboards.
- **Automated Retraining Pipeline:** Schedule periodic retraining as new labeled outcomes accumulate, with versioned model artifacts and a rollback path if a new model underperforms.
- **IBM Cloud Deployment:** Complete the deployment referenced by the footer's “IBM Cloud Ready” badge, moving from the local Flask development server to a managed, publicly accessible, and horizontally scalable hosting environment.
- **Expanded Feature Set:** Incorporate additional signals such as existing debt-to-income ratio or employment tenure to further improve model discrimination beyond the current 11 features.

Appendix A: Project Dependencies

The following Python packages are required to run both the training notebook and the Flask application:

Package	Used For
Flask	Web application framework and routing
numpy	Numeric array construction for the feature vector

Package	Used For
pandas	Dataset loading, cleaning, and exploratory analysis
scikit-learn	LabelEncoder, StandardScaler, train_test_split, and baseline classifiers
xgboost	XGBClassifier — the production model
matplotlib / seaborn	Exploratory visualizations in the training notebook
jupyter	Running and editing the training notebook

A minimal requirements.txt for the Flask application alone:

```
Flask==3.0.3
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.4.2
xgboost==2.0.3
```

Appendix B: Glossary of Terms

Term	Definition
Feature	An individual input column (e.g., ApplicantIncome) used by the model to make a prediction
Label / Target	The outcome the model learns to predict — here, Loan_Status (Approved / Rejected)
Imputation	Filling missing values in a dataset, using the mean for numeric columns and the mode for categorical columns
Label Encoding	Converting categorical text values (e.g., “Male”/“Female”) into integers a model can process
Feature Scaling	Rescaling numeric features (via StandardScaler) to a common range so no

Term	Definition
	single feature dominates due to its raw magnitude
Train/Test Split	Dividing the dataset into a portion used to fit the model (train) and a held-out portion used to evaluate it (test)
Overfitting	When a model learns the training data too precisely (e.g., 100% train accuracy) and generalizes poorly to new data
Precision	Of all applications the model predicted as Approved, the share that were genuinely Approved
Recall	Of all applications that were genuinely Approved, the share the model correctly identified
F1-Score	The harmonic mean of precision and recall, balancing both into a single comparison metric
Confusion Matrix	A table comparing predicted vs. actual outcomes, broken into true/false positives and negatives
Pickle (.pkl)	Python's native serialization format, used here to save the trained model and scaler to disk
XGBoost	“Extreme Gradient Boosting” — an ensemble algorithm that builds trees sequentially, each correcting the errors of the last